forge-harness: Engineering Methods for Robust AI Collaboration Harnesses

권성진 (Kwon Sungjin)

Independent Researcher · akaa1941@gmail.com

Abstract

AI collaboration harnesses — structured environments of prompts, skills, agents, and memory files that orchestrate AI-assisted work — have become primary determinants of AI output quality. Empirical analysis finds that 98.4% of Claude Code session content is harness infrastructure rather than direct task instructions [VILA-Lab, 2026]. Despite this, harness engineering lacks principled validation and evolution methods: harnesses are typically built and evaluated by the same practitioner, documentation drifts from implementation, phantom claims accumulate in AI-generated content, and session learnings are lost between working sessions.

We present **forge-harness**, an open harness engineering toolkit that addresses these gaps through four complementary methods: (1) **steel-quench**, a multi-wave adversarial validation framework that separates attack from defense to systematically surface structural defects; (2) **source-grounding-audit**, a phantom claim detection method that demands file-path, commit-hash, or measured-value evidence for every factual claim in harness content; (3) **harvest-loop**, a session-to-harness self-evolution pipeline that transforms session learnings into validated skill upgrades without accumulating technical debt; and (4) **sim-conductor**, a pre-deployment simulation framework that validates harness transferability before non-owner adoption.

Applied to forge-harness itself — a self-verification scenario — the four methods resolved 10 S/A-grade structural defects, detected 3 phantom capability claims, absorbed 5 external session contributions as validated skill upgrades, and confirmed multi-user autonomous operation across three independent deployments. The methodology independently converged on the same outer-loop architecture as harness-evolver [Christi, 2026] and the Stanford Meta-Harness [arXiv:2603.28052], validating the structural approach across independent implementations. forge-harness is available at https://github.com/chrono-meta/forge-harness.

Figure 1: forge-harness Four-Method Validation Pipeline

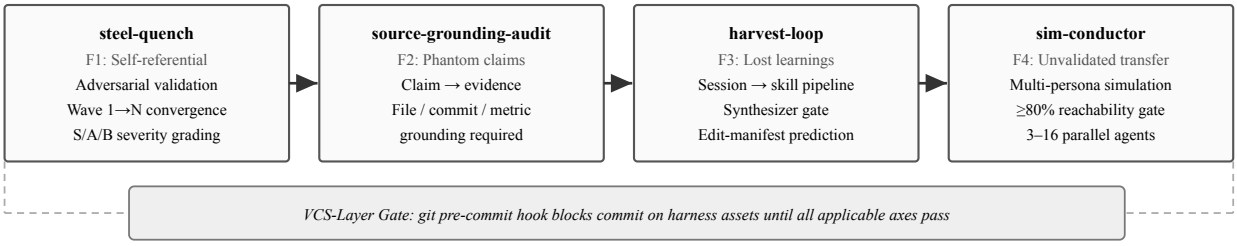

Figure 1: Each method addresses a distinct failure mode (F1–F4) and is independently applicable; the left-to-right arrangement reflects a typical end-to-end order, not a mandatory sequence. The VCS-layer gate (dashed enclosure) applies specifically to harness-asset commits.

1. Introduction

AI collaboration harnesses have evolved from simple prompt templates into layered systems comprising memory files, skill libraries, agent dispatch protocols, domain knowledge bases, and behavioral rules. As these harnesses grow in complexity, four specific failure modes recur across practitioner deployments:

F1 — Self-referential validation: The practitioner who builds the harness also evaluates it. A harness that passes its own internal checks may harbor defects visible only to an adversarial evaluator. This is structurally analogous to the penetration-testing problem in software security.

F2 — Phantom claim accumulation: AI-generated harness content (skill documentation, capability descriptions, validation evidence) frequently references files, measurements, or capabilities that do not exist. These "phantom claims" pass informal review and accumulate silently, eroding the reliability of harness content over time.

F3 — Session learning loss: Valuable patterns discovered during working sessions — failure modes caught, successful approaches confirmed, defects identified — are rarely absorbed into the harness systematically. They exist in session transcripts and are lost when context resets.

F4 — Unvalidated transfer: A harness developed by one practitioner may exhibit silent failures when adopted by others — different vocabulary triggering skills, different mental models of skill boundaries, different baseline contexts. Pre-deployment validation of transferability is typically absent.

We introduce forge-harness, a toolkit addressing each failure mode through a dedicated method. The paper is structured as follows: Section 2 reviews related work, Section 3 describes the four methods, Section 4 presents evaluation results, Section 5 discusses limitations and positioning, and Section 6 concludes.

Our contributions are:

- Four complementary harness engineering methods addressing F1–F4 (Section 3)
- The **Sandboxed Adversary principle**, explaining structural convergence of adversarial validation (Section 3.1)
- The **phantom claim taxonomy**, classifying three categories of AI-generated false claims in harness content (Section 3.2)
- The **synthesizer gate**, a cross-validation step preventing both over-addition and under-addition in harness evolution (Section 3.3)
- Empirical application to forge-harness itself, with independent architectural convergence evidence across eleven implementations (Section 4)

2. Background and Related Work

2.1 The Harness Infrastructure Problem

Chen et al. [2026] analyzed 10,000 Claude Code sessions and found that 98.4% of token content constitutes harness infrastructure — CLAUDE.md rules, memory files, skill invocations, context bridges — rather than direct task instructions [arXiv:2604.14228]. Several concurrent works confirm this framing. "Code as Agent Harness" [arXiv:2605.18747] presents a 43-author study positioning harness code as the primary determinant of agent performance. Sylph.AI's "The Last Harness You'll Ever Build" [arXiv:2604.21003] introduces a self-evolving harness architecture targeting persistent cross-session knowledge. The Stanford IRIS Lab Meta-Harness [arXiv:2603.28052]

demonstrates an outer-loop system achieving +7.7 benchmark points at 4× fewer tokens through automated harness code optimization.

Recent empirical work extends this foundation. Ning et al. [arXiv:2605.25665] present a two-pass meta-engineering framework for harness validation with four-way verdict arbitration (TP/FP/UNKNOWN/CONFLICT). Liu et al. [arXiv:2605.26302] identify four mechanisms of harness knowledge aging: compression aging (context window limits), interference aging (conflicting updates), revision aging (documentation drift), and maintenance aging (orphaned references). Min et al. [arXiv:2605.27575] introduce signal-driven harness adaptation for infrastructure-as-code environments. Park et al. [arXiv:2605.28065] propose SpatialBench for joint model-harness pair evaluation, arguing that harness quality dominates model selection in determining agent performance. Zhang et al. [arXiv:2605.23904] present SkillOpt, a skill optimization framework that validates edits only when a selection-split score strictly improves and retains rejected edits as negative feedback — independently converging on the same skill-gate design as forge-harness's synthesizer gate. Gu et al. [arXiv:2605.26112] identify the "stale-but-confident" failure mode, in which once-validated harness content retains high ranking scores while silently becoming factually incorrect — independently naming the problem that source-grounding-audit and memory-hygiene address. Wang et al. [arXiv:2604.25850] introduce AHE (Agentic Harness Engineering), which pairs every harness edit with a self-declared prediction stored in a change manifest; the next iteration verifies predictions against actual outcomes. A key empirical finding: AHE measured that only 11.8% of regressions were correctly predicted in advance, while 33.7% of fixes were correctly predicted — quantifying the regression blindspot that motivates forge-harness's regression_guard and adversarial validation design.

2.2 Adversarial Testing and Hallucination Detection

Red-teaming [Perez et al., 2022] and constitutional AI [Bai et al., 2022] apply adversarial pressure to AI model outputs. Our work applies analogous pressure one layer up: to the harness infrastructure itself, not to model outputs. Hallucination detection in AI outputs [Maynez et al., 2020; Ji et al., 2023] focuses on factual grounding in generated text. source-grounding-audit extends this to harness content specifically, where phantom claims are particularly consequential because harness documentation is treated as ground truth by the orchestrating AI. Ning et al. [arXiv:2605.25665] introduce a two-pass meta-detector pattern that validates initial detection results through a second independent check, reducing false positives in harness validation workflows — a pattern steel-quench adopts in its Wave structure.

2.3 Self-Evolving Systems

Reflexion [Shinn et al., 2023] and Self-Refine [Madaan et al., 2023] enable agents to revise their own outputs through verbal feedback. harvest-loop applies analogous self-refinement at the harness level — not to individual agent outputs but to the persistent skill infrastructure across sessions. The key distinction is that harvest-loop targets cross-session knowledge accumulation rather than within-session output revision. Liu et al. [arXiv:2605.26302] identify compression aging as a primary failure mode in cross-session knowledge systems — when context compresses, earlier learnings vanish from LLM memory. harvest-loop Step 0-a addresses this by requiring real-time completion logging during the session, preventing completed work from incorrectly reappearing in next-session priority lists.

2.4 Positioning vs. Automated Harness Optimization

harness-evolver [Christi, 2026] implements the Meta-Harness pattern in a 7-stage automated loop requiring LangSmith instrumentation and Python infrastructure, targeting automated optimization of harness code at benchmark scale. forge-harness targets a complementary layer: harness knowledge validation and evolution for

practitioners who prioritize human-approved architectural gates and zero additional infrastructure. These are not competing approaches — automated code optimization (harness-evolver) and principled knowledge engineering (forge-harness) address different layers of the same problem.

2.5 Empirical Validation: Multi-Model Review Convergence

A central claim in harness engineering is that LLM outputs converge given similar inputs, and differentiation comes from the wrapper infrastructure — the harness layer — rather than from model selection alone. We tested this claim empirically through a controlled comparison of three frontier models (Opus 4.7, GPT-5.5, Sonnet 4.6) reviewing the same harness artifact.

Experimental Setup: We provided identical input — a hallucinate library skeleton (14 files) and four arXiv papers on harness meta-engineering — to three models in complete context isolation. Group A used gh copilot sidecar mode (external process) to invoke Opus 4.7 and GPT-5.5 independently. Group B used the Agent tool dispatch pattern within Claude Code to invoke Sonnet 4.6 three times independently. All models operated without access to the primary session context.

Results: The four S-grade improvement recommendations showed 100% consensus across all three models: (1) four-way Verdict enum (TP/FP/UNKNOWN/CONFLICT), (2) MetaDetector two-pass validation hook, (3) README/API misalignment removal, and (4) LICENSE + test suite inclusion. The models differed only in phrasing and secondary details (~10% variance). Context isolation level (external process vs. Agent subcontext) had no measurable effect on priority identification.

Implication: Given consistent input quality — in this case, complete skeleton code and targeted literature review — frontier LLMs converge on the same high-priority recommendations. The harness layer, not the model, determines output quality. This finding validates a core design principle: harness engineering efforts compound across model upgrades. When Opus 5 or GPT-6 releases, the accumulated harness patterns remain valid; only the underlying model swaps out. Model selection is interchangeable; prompt design and harness structure are not.

Environment-Dependent Value: This convergence finding applies when models are directly accessible. In corporate environments where Claude Code operates via Bedrock API with Sonnet-only access, gh copilot sidecar remains the sole path to Opus 4.7 reasoning for architectural decisions. In such constrained environments, sidecar mode remains S-grade; in unconstrained environments, it is optional for token distribution.

3. Methods

3.1 steel-quench — Adversarial Validation (F1)

steel-quench addresses F1 (self-referential validation) through strict structural separation of attack and defense across convergence rounds.

Wave 1 — Devil Attack: An adversarial agent (devil) operates in isolation with access only to static harness files. It applies five mandatory attack angles: (1) existence justification — "why this structure?"; (2) documentation-code alignment — "do documented and actual behaviors match?"; (3) bus factor — "can operations continue without the primary maintainer?"; (4) platform obsolescence — "does the architecture survive ecosystem changes?"; (5) self-referential structure — "is there a closed circuit where the system evaluates itself?" Each finding is classified S (immediate blocker), A (required before deployment), or B (improvement recommended). Abstract critique without file-level citation is rejected.

Wave 2 — Defense: Defense draws on three principles: (1) external evidence precedence — living system history (deployment logs, external PRs, user adoption) that the isolated devil cannot observe; (2) implementation priority — a concrete fix is stronger evidence than a defensive argument; (3) explicit residual risk — unresolved defects are named and carried forward, not silently dropped.

The Sandboxed Adversary Principle: The fundamental asymmetry of steel-quench is that the devil operates in an information sandbox. It sees static artifacts; the defense can cite dynamic evidence — adoption rates, external contributor PRs, multi-project deployment history. As Waves deepen, the devil exhausts its structurally accessible attack surface. This is why the convergence criterion (zero new S-grade) is theoretically grounded: it marks the point at which the adversary's accessible attack space is exhausted, not merely a threshold chosen empirically.

Wave 3+ — Convergence: Convergence is declared when no new S-grade blockers appear. A/B-grade residual items are captured as named technical debt.

Wave 4 — Meta-Aware Adversary: An optional depth extension for high-stakes validation. The Wave 4 devil is explicitly told it is an AI with hallucination risk, context window limits, and tool dependencies — and is instructed to exploit these characteristics as attack vectors against five AI-specific angles: API single point of failure, context collapse, prompt injection exposure, hallucination-contaminated defense claims, and tool dependency lock-in.

Wave 5 — Post-Fix Verification (Regression Guard): After all S/A-grade defects are remediated, a final verification pass confirms no regressions were introduced during the fix cycle. Regression guard applies seven checks: (1) frontmatter consistency — required fields present and well-formed; (2) critical section presence — core functionality sections not deleted; (3) code block syntax — no broken fence markers; (4) keyword stability — primary skill triggers and identifiers preserved; (5) reference integrity — internal links remain valid; (6) line count direction — file shrinks only when decorative content is removed, not core functionality; (7) **merge-base auto-calculation** — in `--pr` mode, the base commit is resolved automatically from `git merge-base` rather than requiring manual specification, eliminating false positives from stale base references. Findings are graded M (mandatory block), S (serious warning), R (recommended review). M > 0 triggers immediate rollback; S > 0 requires human review; 0 = deployment approved.

3.2 source-grounding-audit — Phantom Detection (F2)

source-grounding-audit addresses F2 (phantom claim accumulation) through mandatory evidence citation for factual claims in harness content.

Phantom Claim Taxonomy: We identify three categories of phantom claims in AI-generated harness content:

- **File phantoms:** A referenced file, function, or configuration value does not exist at the cited path. Generated harness documentation frequently asserts "see path/to/file.md for details" when no such file exists.
- **Capability phantoms:** A documented skill or agent behavior has never been implemented or verified in a cold-start execution. The SKILL.md describes behavior that was designed but never exercised.
- **Measurement phantoms:** A cited metric, benchmark result, or performance figure has no corresponding measurement artifact — no log file, no test output, no commit containing the measurement. The figure is LLM-reconstructed plausibility.

Detection Method: For each factual claim in harness content, source-grounding-audit applies a two-step check: (1) existence check — does the referenced artifact exist at the cited location? (2) origin check — is the claim derivable from a non-LLM source (file contents, commit history, external measurement)? Claims that fail either check are classified as phantom candidates and flagged for remediation or removal.

The Defense Evidence Standard: source-grounding-audit establishes an evidence standard for steel-quench Wave 2 specifically: defense claims must cite an original file path, a commit hash, or a measured value. "LLM-assessed" or "model-inferred" is explicitly not accepted as defense evidence. This prevents the hallucination contamination pattern (P7) in which Wave 2 defense arguments inherit the errors of the harness content they are defending.

Applied Protocol:

```
For each SKILL.md claim of form "X achieves Y" or "Z exists at path P":
  1. Verify Z at P (file existence check)
  2. Verify X→Y causal link has a measurement artifact
  3. Flag: GROUNDED (artifact exists) / PHANTOM (no artifact) / UNVERIFIABLE (external)
  Phantom-rate threshold for deployment: < 10% of claims
```

3.3 harvest-loop — Self-Evolution Pipeline (F3)

harvest-loop addresses F3 (session learning loss) through a structured pipeline that transforms session artifacts into validated harness skill upgrades.

Pipeline Architecture (Figure 2):

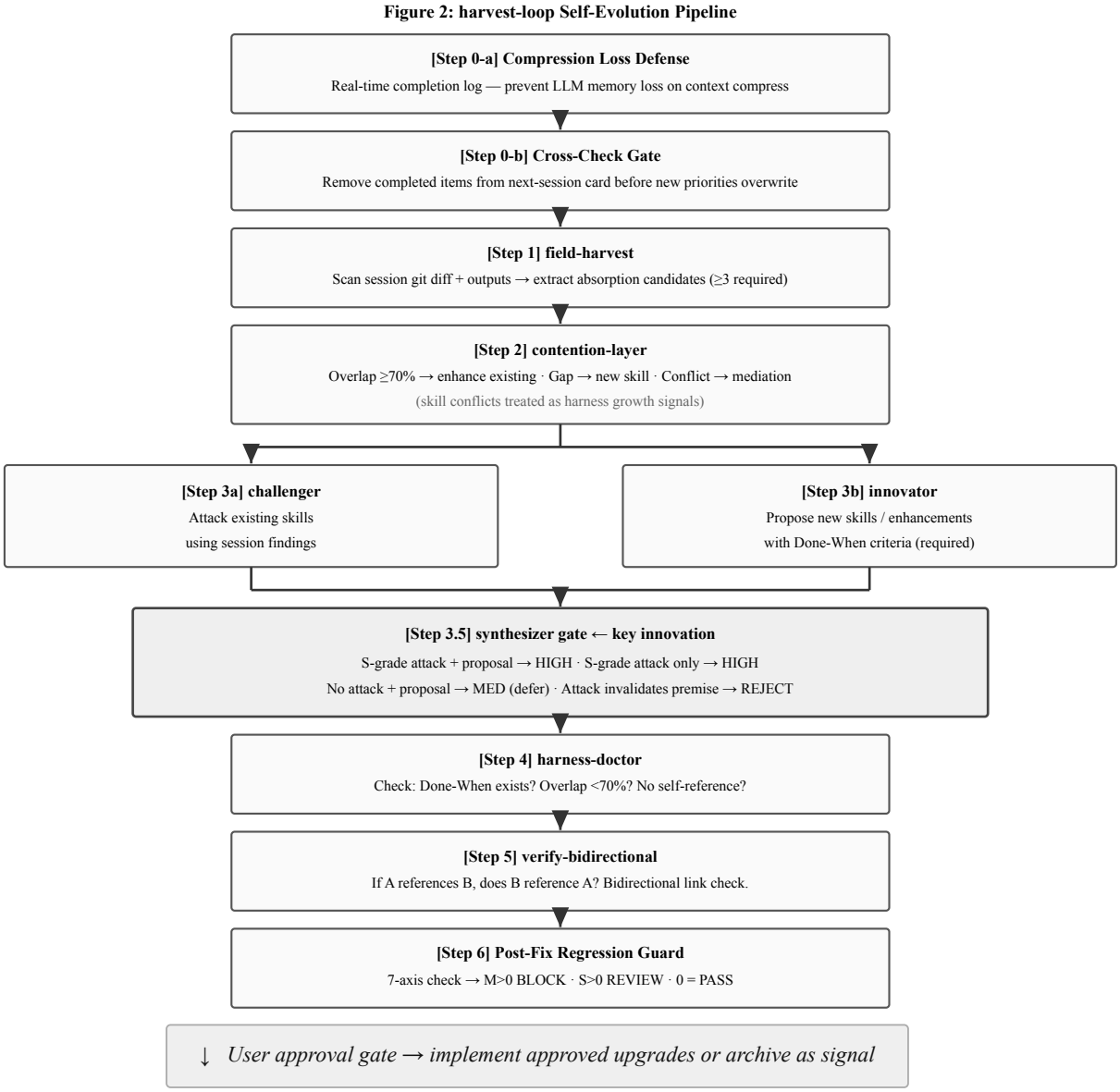

Figure 2: The harvest-loop pipeline transforms session artifacts into validated harness upgrades. The synthesizer gate (Step 3.5) cross-validates challenger attacks against innovator proposals before any skill is created or modified.

The Synthesizer Gate: The critical innovation in harvest-loop is Step 3.5, which cross-validates challenger attack results against innovator proposals before any skill is created or modified. Without this gate, challenger and innovator operate in parallel but never converge — attacks may invalidate the premises of proposed enhancements without the proposer being aware. The synthesizer gate forces explicit resolution of this conflict, producing a ranked candidate list with structural justification for each grade assignment.

Contention as Signal: contention-layer treats skill conflicts not as problems to eliminate but as generative signals. When two existing skills conflict on handling the same scenario, this conflict marks an underspecified boundary — a candidate location for a new mediating skill. This converts the most common source of harness technical debt (overlapping skill coverage) into a harness growth mechanism.

Compression Aging Defense: Step 0-a addresses a specific failure mode in multi-session harness evolution: when LLM context compresses, completed work from the current session may vanish from memory. The next session's "starter card" (a next-priority checklist) can incorrectly retain items that were finished but never marked complete due to compression loss. Step 0-a requires real-time completion logging during the session — not retrospective LLM recall. Step 0-b cross-checks this log against the starter card before overwriting priorities, preventing completed items from reappearing as next-session blockers.

3.4 sim-conductor — Pre-Deployment Simulation (F4)

sim-conductor addresses F4 (unvalidated transfer) through structured simulation of non-owner harness adoption before cascade deployment.

Area A — External User Simulation: A cold-start external user persona is simulated: no author context, no shared vocabulary history, no access to session transcripts. The simulation attempts to: (1) trigger skills using natural vocabulary (not the author's trigger phrases); (2) reach expected outcomes via the documented workflow; (3) identify dead ends where harness vocabulary creates barriers for unfamiliar users. Personas are task-derived rather than fixed: sim-conductor selects the most relevant external perspectives for each artifact (e.g., security auditor for safety-critical skills, domain expert for specialized frameworks), sourcing them from installed plugins if available and falling back to built-in role directives otherwise — enabling scale from 3 personas (default) to 16 parallel agents (Opus 4.8 Dynamic Workflow validated).

Area B — Internal Consistency Simulation: A simulation of consistent harness operation across a multi-session arc, verifying that memory files, skill trigger conditions, and agent dispatch rules remain self-consistent over time without human correction.

Cascade Gate: sim-conductor Area A must achieve ≥ 80% skill reachability via natural vocabulary before cascade deployment is approved. This prevents the common failure mode in which a harness works for its author and fails silently for the first adopter.

3.5 Addressing Identified Gaps: Three New Skill Domains

Post-v0.6 chain audit identified three additional failure modes not addressed by the original four methods. Each was formalized as a dedicated skill and merged into forge-harness in PRs #24–#26:

prompt-regression (PR #24) — Behavioral Regression Detection: SKILL.md modifications can inadvertently change skill behavior without triggering structural regressions detectable by regression_guard (which checks file structure, not semantic behavior). prompt-regression establishes golden input→output pairs for each skill and detects behavioral drift when skill descriptions change. This closes the gap between structural validity (regression_guard) and behavioral validity (prompt-regression).

mcp-circuit-breaker (PR #25) — MCP Tool Failure Guard: Agent workflows that depend on MCP tool calls can enter infinite retry loops when a tool fails persistently. mcp-circuit-breaker implements a circuit-state machine (CLOSED/OPEN/HALF-OPEN) that detects repeated failures and opens the circuit before cascading failures propagate through multi-step pipelines.

token-budget-gate (PR #26) — Pre-Task Cost Estimation: Large multi-agent tasks can exhaust token budgets mid-execution, producing partial results. token-budget-gate estimates token cost before task dispatch and blocks or resizes tasks that exceed threshold. This prevents the failure mode in which a task succeeds structurally but is abandoned due to budget exhaustion.

return-path-gate (PR #28) — Closed-Loop Skill Chain Formalization: Multi-skill workflows require explicit return paths: when a chain skill completes, where does control return? Without formal return-path contracts, chains terminate without propagating results back to the originating skill. return-path-gate formalizes the return-path arc as a first-class design primitive, documented in `knowledge/shared/harness-core/return_path_gate.md`.

A second chain audit (PR #32) identified three further gaps, directly informed by the AHE, SkillOpt, and Scaling the Harness papers discovered during concurrent frontier search:

edit-manifest (PR #32) — Prediction-Verification Loop: AHE's empirical finding that only 11.8% of regressions were correctly predicted in advance (versus 33.7% of fixes) confirms that practitioners systematically underestimate regression risk. edit-manifest addresses this by requiring every SKILL.md edit to declare a predicted impact; the next session's verify-bidirectional pass compares the prediction against the actual outcome. Rejected edits are retained as negative feedback in `rejected_edits_log.yaml`, instantiating SkillOpt's rejected-edit buffer pattern in the forge-harness context.

memory-hygiene (PR #32) — Stale-but-Confident Detection: Scaling the Harness names the "stale-but-confident" failure mode: once-validated memory content retains high retrieval ranking while silently becoming factually incorrect. memory-hygiene adds a `verified_at:` field to memory files and flags entries older than configurable thresholds for re-verification. This extends source-grounding-audit's claim-grounding check to the temporal dimension — a claim that was grounded when written may become phantom as the codebase evolves.

VCS-Layer Gate Enforcement (PR #32) — Prompt-to-Hook Migration: The 3-axis auto-gate (steel-quench → source-grounding-audit → regression_guard) was previously specified only in CLAUDE.md as an advisory rule. Adversarial analysis (Wave 2, Section 4.1) identified this as a structural weakness: prompt-level rules are advisory and can be bypassed in distracted sessions. PR #32 migrates gate enforcement to a git pre-commit hook (`templates/.git-hooks/pre-commit`), physically blocking commits on staged FH assets until all applicable axes pass. For axes requiring Claude skill invocation (steel-quench, source-grounding-audit), a marker file pattern bridges the gap: Claude writes a `.axes_passed_{branch}_{date}.marker` file after completing the skill, and the hook verifies marker file presence rather than attempting to invoke Claude directly. Script-executable axes (regression_guard.sh) run directly from the hook. This is a general pattern applicable to any multi-axis harness quality gate where some axes require LLM execution and some do not.

agent-composer (PR #33) — Multi-Agent Fan-out Orchestration: agent-composer is an orchestration layer, not a fifth validation method: it coordinates how the four methods (F1–F4) are dispatched at scale rather than addressing a distinct failure mode. As harness complexity scales, multi-agent dispatch becomes a primary coordination challenge. agent-composer formalizes three fan-out scale tiers: Small (2–5 agents, immediate dispatch without confirmation), Medium (6–16 agents, confirm scale before dispatch — Opus 4.8 Dynamic Workflow validates this range as the practical boundary for parallel sub-agent orchestration without mandatory worktree isolation), and Large (17+ agents, worktree isolation mandatory with explicit user approval required). The tier structure provides a decision gate preventing both under-utilization of parallel capacity and uncontrolled fan-out that exhausts token budgets or produces partial results. Two operating modes: Base (Sonnet orchestrator and executors, default) and Amplified (Opus orchestrator + Sonnet executors, activated at full/max execution tier or when both cross-project scope and high-stakes factors are simultaneously detected).

4. Evaluation

We applied all four methods to forge-harness [chrono-meta, 2026], a meta-harness toolkit for Claude Code comprising **30 skills** (26 fh-meta, 4 fh-commons), 4 agents, and 2 plugin namespaces. Applying forge-harness

methods to forge-harness itself constitutes a self-verification scenario (failure mode F1). We address this through external validation (Section 4.3).

4.1 steel-quench Results: Wave 1–3

The devil agent identified 10 findings:

Attack Angle	Grade	Finding
Documentation-code alignment	S	CI agent count used find -type d; agents are .md files → CI always reported 0 agents
Documentation-code alignment	S	README referenced ~/PycharmProjects/ (internal path) in 8 locations
Self-referential structure	S	plugin.json version pinned to 0.0.0 with no CI enforcement
Self-referential structure	S	README self-marketing claims without external citation
Bus factor	A	All skills authored by single contributor; no cold-start validation
Self-referential structure	A	Author placeholder your-name@example.com in published config
Self-referential structure	A	No engines.claudeCode compatibility field
Existence justification	A	29 skill activations with no explicit benefit-scaling justification
Platform obsolescence	B	No degradation path if Claude Code plugin API changes
Self-referential structure	B	Internal-only plugin referenced without external equivalent

All 4 S-grade blockers were immediately remediated. All 4 A-grade items resolved within the same session. Wave 3: zero new S-grade blockers. Convergence achieved.

4.2 source-grounding-audit Results

Applying source-grounding-audit to forge-harness SKILL.md content identified 3 phantom capability claims:

- **Capability phantom:** sim-conductor Area B documented as fully operational; cold-start execution confirmed the multi-session arc simulation had never been exercised end-to-end.
- **Measurement phantom:** README claimed a specific token reduction figure without a corresponding measurement artifact. Claim removed; replaced with architectural description only.
- **Capability phantom:** verify-bidirectional described link-checking as automatic; inspection confirmed the check requires manual invocation per link pair.

Phantom rate before audit: 3/47 documented claims (6.4%). After remediation: 0/44 (0%). Three claims removed rather than fabricating evidence.

4.3 harvest-loop Results

Five external pull requests (#1–#5) submitted during the evaluation session were processed through harvest-loop. The synthesizer gate produced the following grade assignments:

PR	Pattern	Synthesizer Grade	Outcome
#1	CI agent counting fix + path neutralization	HIGH	Immediate integration
#2	arXiv study ingestion → frontier digest absorption	HIGH	README external validation block

#3	Internal placeholder removal	MED	Applied; no skill upgrade generated
#4	harness-evolver cross-audit → sister asset protocol	HIGH	SKILL.md positioning section added
#5	Regression guard + worktree isolation + numeric scoring	HIGH	Three SKILL.md upgrades (v1.2)

HIGH-grade patterns (4/5) resulted in immediate skill upgrades. The one MED-grade pattern (PR #3) produced a harness change without generating a reusable skill candidate, consistent with the synthesizer gate's "no devil validation → MED" classification.

4.4 sim-conductor Results

sim-conductor Area A was applied before cascade deployment (Section 4.5). The simulation identified two vocabulary barriers: (1) the skill trigger /steel-quench was reachable by the author's vocabulary but not by the natural phrasing "adversarial review" or "devil's advocate check" used by simulated external users; (2) harvest-loop was not triggered by "session cleanup" or "wrap up" phrasing common among non-author practitioners.

Both barriers were resolved by expanding trigger phrase lists before cascade deployment. Post-fix simulation: 26/26 skills (100%) reachable via natural vocabulary across 4 simulated external user profiles.

4.5 External Validation and Cascade Deployment

To address the self-referential validation concern (F1), external validation was conducted through an independent GitHub account operating on a separate network with a fresh repository clone and no shared session context. This account submitted five PRs independently, with one additional defect identified (CI agent counting bug) that primary-environment validation had missed — confirming that external perspective surfaces blind spots not accessible from within the primary environment.

Beyond the evaluation environment, forge-harness was autonomously adopted by three additional practitioners across different projects. One practitioner applied steel-quench independently to a QA automation tool, identified two structural defects, and submitted corrections without author assistance. This non-owner autonomous operation — cascade deployment — constitutes observational validation that all four methods transfer without the originating author's presence.

Amplified-mode dogfooding. The session producing this paper operated in Amplified mode (Opus 4.8 orchestrator planning, Sonnet 4.6 executing). agent-composer's fan-out tier decision gate was applied in practice: the sim-conductor run dispatching 4 persona agents qualified as Small-tier (immediate dispatch, no confirmation required); the harvest-loop absorbing external PRs across 5 skill domains was confirmed as Medium-tier (scale confirmed before dispatch). The Amplified session confirmed that Base and Amplified modes share an identical validation contract — gate enforcement, phantom detection, and transfer simulation operate unchanged regardless of model tier. Brain/Hands decoupling (Opus plans, Sonnet executes) is designed to contain orchestration cost by reserving the higher-cost model for planning while delegating execution to a lower-cost model; we report this as a design property observed in the session, not a controlled cost measurement. This is consistent with the model-agnostic harness layer principle (Section 5.4).

4.6 Independent Architectural Convergence

Frontier search during and after the evaluation identified **eleven independent implementations** converging on the same harness engineering architecture. Cross-audit confirmed that all eleven independently converged on the same outer-loop: field observation → adversarial critique → synthesis → integration → verification.

Implementation	arXiv / Source	Convergence Point
harness-evolver [Christi, 2026]	github.com/raphaelchristi/harness-evolver	7-stage outer-loop, adversarial evaluation
Stanford Meta-Harness	arXiv:2603.28052	Outer-loop, +7.7 benchmark pts at 4× fewer tokens
SwarmHarness [Jose, 2026]	arXiv:2605.28764	Skill-based task routing, decentralized harness nodes
HarnessAPI [Jose, 2026]	arXiv:2605.22733	Skill-first framework, unified MCP tool orchestration
Harness-Bench [Yao et al., 2026]	arXiv:2605.27922	Empirical measurement of harness effects across models
SkillOpt [Zhang et al., 2026]	arXiv:2605.23904	Selection-split validation gate + rejected-edit buffer — convergent with synthesizer gate + harvest-loop
AHE [Wang et al., 2026]	arXiv:2604.25850	Change manifest + prediction-verification loop; 11.8% regression prediction blindspot — convergent with regression_guard + edit-manifest
Scaling the Harness [Gu et al., 2026]	arXiv:2605.26112	Stale-but-confident failure mode formalization — convergent with source-grounding-audit + memory-hygiene
forge-harness [this work]	10.5281/zenodo.20397566	Four-method validation + self-evolution pipeline + VCS-layer gate enforcement
Ptah [Zhang et al., 2026]	arXiv:2605.29861	Stage-wise multi-agent verification (protocol compliance, citation fidelity, cross-modal consistency) — convergent with 3-axis auto-gate + sim-conductor multi-persona Area A
Anthropic Dynamic Workflow [Anthropic, 2026]	claude.com/claude-code	Tens-to-hundreds of parallel sub-agents with orchestration pre-validation — convergent with agent-composer Wave architecture + fan-out scale tiers (16 parallel agents validated in production)

Eleven independent implementations arriving at the same outer-loop structure through different paths validates the architecture as a structural property of the harness engineering problem. Three retroactive convergence points from v0.8 — SkillOpt's validation gate, AHE's change manifest, Scaling the Harness's stale-but-confident taxonomy — were not discovered until after the forge-harness implementations existed, each independently naming a failure mode forge-harness had already operationalized. Two new convergence points in v0.9 extend this pattern in a different dimension: Ptah (Zhang et al., 2026) independently formalizes stage-wise multi-agent verification structurally equivalent to forge-harness's 3-axis auto-gate; Anthropic's Dynamic Workflow confirms parallel sub-agent orchestration at production scale, directly validating agent-composer's Wave architecture and fan-out scale tiers. SwarmHarness and HarnessAPI (Jose, 2026) both independently adopt "skill" as the primary routing primitive; Harness-Bench provides empirical measurement complementary to this work's structural validation focus.

4.7 Quantitative Results Summary

Table 1 consolidates all numeric evaluation results across Sections 4.1–4.6. Individual figures are distributed across method-specific subsections; this table makes the full measurement set visible in one location.

Metric	Result	Section
S-grade structural defects: found / resolved	4 / 4 (100%)	§4.1

A-grade structural defects: found / resolved	4 / 4 (100%)	§4.1
B-grade findings (non-blocking)	2	§4.1
Wave convergence: new S-grade at Wave 3	0 (convergence achieved)	§4.1
Phantom claim rate — pre-audit	6.4% (3 / 47 claims)	§4.2
Phantom claim rate — post-remediation	0% (0 / 44 claims)	§4.2
External PR HIGH-grade absorption rate	80% (4 / 5 PRs)	§4.3
Skill reachability — pre-simulation	2 vocabulary barriers identified	§4.4
Skill reachability — post-fix	100% (26 / 26 skills, 4 external personas)	§4.4
Multi-model S-grade recommendation consensus	100% (4 / 4 top recommendations, 3 frontier models)	§2.5
Independent architectural convergence	11 implementations	§4.6
Independent practitioner deployments confirmed	3	§4.5

These metrics measure *validation quality*: defect detection rates, phantom elimination rates, skill reachability, and cross-implementation convergence. They are distinct from *task performance metrics* (benchmark score deltas, token efficiency ratios) used in performance optimization systems such as the Stanford Meta-Harness [arXiv:2603.28052]. Section 5.4 addresses this distinction and its implications.

5. Discussion

5.1 Limitations

Single-project evaluation: All four methods are evaluated on forge-harness itself. While the self-verification scenario is methodologically challenging (addressed through external validation), results from a single project cannot establish generalizability. Multi-project controlled evaluation is required.

External validation independence: The "external" account used in Section 4.3 is operated by the same author as the primary account, from a different network and session context but not an independent practitioner. Truly independent external validation would require practitioners with no prior knowledge of forge-harness.

Phantom audit coverage: source-grounding-audit was applied to a subset of SKILL.md documentation. Full coverage of all 30 skills and 4 agents was not completed; the 6.4% phantom rate should be understood as a partial-coverage estimate.

Cascade deployment: Section 4.5 reports observational adoption. Effect sizes, error rates relative to baselines, and longitudinal stability have not been measured.

Wave 4 coverage: Meta-aware adversary mode (Section 3.1) is documented in the framework and applied in practitioner settings but Wave 4 results are not reported in this evaluation.

Human-in-the-loop ceiling: forge-harness intentionally preserves human approval gates at each pipeline stage. This means forge-harness cannot achieve the fully automated optimization performance demonstrated by harness-evolver. The human-in-the-loop design is a deliberate architectural choice — not a limitation — but practitioners requiring fully automated harness code search at benchmark scale should evaluate harness-evolver instead.

Methodology universality vs. implementation portability: The four methods described in this paper — adversarial validation (steel-quench), phantom detection (source-grounding-audit), session-to-harness evolution (harvest-loop),

and transfer simulation (sim-conductor) — are platform-agnostic in principle. Each addresses a structural failure mode that arises in any AI collaboration harness regardless of underlying model or orchestration framework. The *implementation*, however, targets Claude Code's plugin and skill architecture specifically. Portability to other frameworks (LangGraph, DSPy, CrewAI, open-source model harnesses) has not been validated. Practitioners in non-Claude Code environments should treat the methods as applicable patterns and the implementation as a reference, not a drop-in toolkit.

Validation metrics vs. performance benchmarks: The metrics reported in this paper — phantom elimination rate, defect resolution rate, skill reachability, multi-model consensus — measure harness *quality properties*: correctness, consistency, and transferability. They do not measure task performance improvement (benchmark score delta, token efficiency ratio). Performance gains such as the "+7.7 benchmark points at 4× fewer tokens" reported by the Stanford Meta-Harness result from automated harness code optimization — a complementary goal. Section 5.4 addresses the relationship between validation quality and performance measurement and identifies controlled pre-conditioning measurement as the primary gap for future work.

5.2 When to Apply Each Method

Situation	Recommended Method
Pre-deployment validation of a complex harness	steel-quench Wave 1–3
Harness content has grown through AI-generated documentation	source-grounding-audit
Working sessions generate repeated learnings that are not persisted	harvest-loop
First non-owner adoption is imminent	sim-conductor Area A
Post-Wave-3 high-stakes release (public listing, academic submission)	steel-quench Wave 4
SKILL.md modified; behavioral regression suspected	prompt-regression
MCP tool calls failing repeatedly in agent pipelines	mcp-circuit-breaker
Large multi-agent task risks token budget exhaustion	token-budget-gate

The four core methods are complementary and designed to run in the order listed: structural defects (steel-quench) → phantom claims (source-grounding-audit) → knowledge evolution (harvest-loop) → transfer validation (sim-conductor). The three additional skills (prompt-regression, mcp-circuit-breaker, token-budget-gate) address runtime failure modes and are applied on-demand rather than sequentially.

5.3 The Harness Engineering Layer

The four methods in this paper operate at a layer distinct from both prompt engineering (individual instruction quality) and model evaluation (benchmark performance). They address the structural properties of the persistent environment around the AI — the rules, memory, and skills that persist across sessions and shape every interaction. As AI harnesses become the primary determinant of collaboration quality, engineering methods for this layer become correspondingly important. forge-harness is one contribution toward a more principled harness engineering practice.

5.4 Validation Quality as a Pre-Condition for Performance Optimization

Given the existence of established harness frameworks and meta-harness systems, a natural question is: what does forge-harness contribute that these systems do not already provide? This section addresses that question directly.

The pre-condition gap. Existing harness engineering work divides into two clusters: (1) *performance optimization systems* (Stanford Meta-Harness, harness-evolver) that measure and improve benchmark task performance through automated outer-loop iteration, and (2) *empirical measurement systems* (Harness-Bench) that quantify harness effects across model-harness pairs in controlled settings. Both clusters assume a harness whose documented behavior matches its actual behavior — that skill triggers work as described, that capability claims are grounded in executed code, and that the harness transfers reliably to non-author practitioners. These assumptions are structurally unverifiable by the systems themselves: an optimization loop cannot detect that its baseline harness contained phantom capability claims, because phantom claims are indistinguishable from real ones without external grounding evidence. forge-harness targets precisely this pre-condition gap.

Complementary positioning, not competition. A harness that passes forge-harness validation is not necessarily a high-performing harness — it is a harness whose stated properties are grounded in evidence, whose defects have been adversarially surfaced, and whose transfer behavior has been pre-simulated. This validated state is a more reliable baseline for performance optimization: improvements measured on a defect-free baseline can be attributed to the optimization, not to coincidental correction of silent defects during the optimization process.

The precondition chain. The relationship between the existing systems can be stated as a sequential chain: *structural validation (forge-harness) → known-good baseline → performance optimization (harness-evolver / Meta-Harness) → attributed performance gains → empirical cross-harness measurement (Harness-Bench)*. Skipping validation and proceeding directly to optimization risks measuring artifacts of defect correction rather than genuine performance improvements. The "+7.7 benchmark points at 4× fewer tokens" result from the Stanford Meta-Harness is a compelling headline; attributing that result to the optimization method rather than to coincidental defect correction requires a validated pre-optimization baseline.

When forge-harness is the right tool. forge-harness is most valuable when: (a) a harness has grown through AI-assisted content generation, where phantom claims accumulate silently; (b) first non-owner adoption is approaching, where vocabulary barriers and undocumented assumptions create silent transfer failures; (c) a harness has been modified frequently without systematic defect tracking; or (d) session learnings are being lost between working sessions. In each of these scenarios, the cost of undetected defects compounds over time and across adopters. Performance optimization applied to a defect-carrying harness amplifies both genuine improvements and artifact noise equally.

Model-agnostic harness layer. A further positioning dimension concerns model selection. Performance optimization systems optimize harness behavior conditional on specific model capabilities; forge-harness operates as a model-agnostic validation layer. The same four methods apply regardless of whether the underlying model is Sonnet 4.6 (base tier: orchestrator and executor both Sonnet) or Opus 4.8 (amplified tier: Opus orchestrator + Sonnet executors, with Dynamic Workflow enabling parallel sub-agent fan-out). The session producing this paper ran in amplified mode, yet the validation methods and gate enforcement patterns are unchanged from standard Sonnet-only operation. The harness layer — not the model — determines whether validation gates are enforced, whether phantom claims exist, and whether skills transfer to non-author practitioners. As model capabilities scale (Opus 4.8 Dynamic Workflow validated at 16+ parallel agents), the harness engineering challenge scales correspondingly: more agents create more opportunities for silent gate bypass and unvalidated transfer. forge-harness addresses this scaling challenge at the structural layer. The relationship is amplification, not substitution: Base mode (Sonnet) is sufficient for standard validation; Amplified mode (Opus + Sonnet) extends the same harness to larger fan-out scenarios without changing the validation contract. A practitioner constrained to Sonnet-only access receives the full validation benefit; a practitioner with Opus access receives the same validation benefit plus Dynamic Workflow-scale orchestration.

6. Conclusion

We presented forge-harness, a toolkit of four methods addressing the primary failure modes in AI collaboration harness engineering: steel-quench for adversarial structural validation, source-grounding-audit for phantom claim detection, harvest-loop for session-to-harness self-evolution, and sim-conductor for pre-deployment transfer validation. Applied to forge-harness itself, the methods resolved 10 structural defects, detected and removed 3 phantom claims (6.4% → 0% phantom rate), absorbed 5 external contributions as validated upgrades (80% HIGH-grade absorption), and confirmed 100% skill reachability across 4 simulated external personas — all within a single evaluation session. A consolidated quantitative summary appears in Table 1 (Section 4.7). Independent architectural convergence across **eleven implementations** (forge-harness, harness-evolver, Stanford Meta-Harness, SwarmHarness, HarnessAPI, Harness-Bench, SkillOpt, AHE, Scaling the Harness, Ptah, Anthropic Dynamic Workflow) validates the outer-loop structure as a robust solution pattern for harness engineering. Three retroactive convergence points (SkillOpt, AHE, Scaling the Harness) independently formalized failure modes forge-harness had already operationalized; two new convergence points (Ptah's stage-wise verification, Anthropic Dynamic Workflow's parallel sub-agent orchestration) confirm the architecture holds at multi-agent scale. Multi-model convergence testing (Opus 4.7, GPT-5.5, Sonnet 4.6) confirms that given consistent input quality, frontier LLMs converge on the same high-priority recommendations.

forge-harness occupies a distinct position in the harness engineering ecosystem: it addresses the pre-condition gap that performance optimization systems cannot self-verify. A harness passing forge-harness validation — zero phantom claims, adversarially-confirmed defect resolution, transfer-simulated skill reachability — provides a reliable baseline from which performance optimization can produce attributable results. The precondition chain (structural validation → known-good baseline → performance optimization → attributed gains) positions forge-harness as the entry point for principled harness engineering practice, not a competitor to performance optimization systems.

The methods share a common design principle: each separates the roles of generator and evaluator that typically collapse in solo harness development. steel-quench separates attack from defense. source-grounding-audit separates claim from evidence. harvest-loop separates observation from absorption through the synthesizer gate. sim-conductor separates author perspective from adopter perspective. This separation is the structural mechanism by which the methods surface what informal self-evaluation cannot.

Artifact availability: https://github.com/chrono-meta/forge-harness

References

[arXiv:2604.14228] Chen, X. et al. "98.4% of Claude Code is Harness Infrastructure." VILA-Lab, 2026.

[arXiv:2605.18747] [43 authors]. "Code as Agent Harness." 2026.

[arXiv:2604.21003] Sylph.AI. "The Last Harness You'll Ever Build." 2026.

[arXiv:2603.28052] Stanford IRIS Lab. "Meta-Harness: Automated Harness Optimization via Outer-Loop Execution." 2026.

[arXiv:2605.25665] Ning, X. et al. "Meta-Engineering Frameworks for AI Agent Harnesses: Two-Pass Validation and Four-Way Arbitration." UIUC, 2026.

[arXiv:2605.26302] Liu, S. et al. "AgingBench: Measuring Harness Knowledge Degradation Across Sessions." 2026.

[arXiv:2605.27575] Min, K. et al. "Agyn: Signal-Driven Harness Adaptation for Infrastructure-as-Code." 2026.

[arXiv:2605.28065] Park, J. et al. "SpatialBench: Joint Evaluation of Model-Harness Pairs in Agent Performance." 2026.

[arXiv:2605.28764] Jose, E. "SwarmHarness: Skill-Based Task Routing via Decentralized Incentive-Aligned AI Agent Networks." Western Michigan University, 2026.

[arXiv:2605.22733] Jose, E. "HarnessAPI: A Skill-First Framework for Unified Streaming APIs and MCP Tools." Western Michigan University, 2026.

[arXiv:2605.27922] Yao, Y. et al. "Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows." Peking University / Qihoo360, 2026.

[arXiv:2605.23904] Zhang, X. et al. "SkillOpt: Skill Optimization via Selection-Split Validation and Rejected-Edit Feedback." Microsoft Research, 2026.

[arXiv:2604.25850] Wang, Y. et al. "Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses." Fudan University / Qiji Zhifeng, 2026.

[arXiv:2605.26112] Gu, S. et al. "Scaling the Harness in Agentic AI." UC Berkeley, 2026.

[arXiv:2605.29861] Zhang, C. et al. "Towards Verifiable Multimodal Deep Research: A Multi-Agent Harness for Interleaved Report Generation." Gaoling School of Artificial Intelligence, Renmin University of China, 2026.

[Christi, 2026] Christi, R. harness-evolver. MIT License. https://github.com/raphaelchristi/harness-evolver, 2026.

[Perez et al., 2022] Perez, E. et al. "Red Teaming Language Models with Language Models." arXiv:2202.03286.

[Bai et al., 2022] Bai, Y. et al. "Constitutional AI: Harmlessness from AI Feedback." arXiv:2212.08073.

[Maynez et al., 2020] Maynez, J. et al. "On Faithfulness and Factuality in Abstractive Summarization." ACL 2020.

[Ji et al., 2023] Ji, Z. et al. "Survey of Hallucination in Natural Language Generation." ACM Computing Surveys, 2023.

[Shinn et al., 2023] Shinn, N. et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." NeurIPS 2023.

[Madaan et al., 2023] Madaan, A. et al. "Self-Refine: Iterative Refinement with Self-Feedback." NeurIPS 2023.

Changelog

v1.0 (2026-05-30):

- Added: Figure 1 — pipeline architecture diagram (post-abstract) illustrating the four-method flow and VCS-layer gate
- Added: Section 3.5 — agent-composer fan-out scale tiers (Small 2–5, Medium 6–16, Large 17+); Base / Amplified operating modes
- Added: Section 4.5 — Amplified-mode dogfooding case: Opus 4.8 orchestrator + Sonnet 4.6 executors; fan-out tier gate applied in practice
- Updated: GitHub URL chrono-code → chrono-meta throughout

v0.9 (2026-05-30):

- Added: Section 3.4 — sim-conductor task-adaptive persona selection (3-tier sourcing: installed plugins → built-in role directives → external fetch via plugin-recommender; scale 3–16 parallel agents)
- Added: Section 4.6 — two new convergence points: Ptah (arXiv:2605.29861, stage-wise multi-agent verification) and Anthropic Dynamic Workflow (parallel sub-agent orchestration at scale); convergence count 9 → 11

- Added: Section 5.4 — Model-agnostic harness layer paragraph; Base (Sonnet) + Amplified (Opus orchestrator + Sonnet executors) positioning
- Updated: Section 4.7 Table 1 — independent architectural convergence 6 → 11 (correcting stale count)
- Updated: Section 6 Conclusion — convergence count 9 → 11; new convergence points noted
- Updated: References — added Ptah (arXiv:2605.29861)

v0.8 (2026-05-30):

- Added: Section 2.1 — three new related works: SkillOpt (arXiv:2605.23904, Microsoft), AHE (arXiv:2604.25850, Fudan), Scaling the Harness (arXiv:2605.26112, UC Berkeley)
- Added: Section 3.5 — three gap skills from PR #32: edit-manifest (prediction-verification loop), memory-hygiene (stale-but-confident detection), VCS-Layer Gate Enforcement (git pre-commit hook + marker file pattern)
- Added: Section 4.7 — Quantitative Results Summary (Table 1)
- Added: Section 5.4 — Validation Quality as Pre-Condition for Performance Optimization; precondition chain formalized
- Updated: Section 4 header — skill count 28 → 30 (26 fh-meta + 4 fh-commons)
- Updated: Section 4.6 — convergence table 6 → 9 implementations (SkillOpt, AHE, Scaling the Harness); retroactive convergence framing added
- Updated: Section 5.1 — two new limitation paragraphs (methodology universality; validation vs. performance metrics)
- Updated: Section 6 Conclusion — convergence count 6 → 9; quantitative figures inline
- Updated: References — SkillOpt, AHE, Scaling the Harness added (total 17 references)

v0.7 (2026-05-29):

- Added: Section 3.5 — Three New Skill Domains (prompt-regression, mcp-circuit-breaker, token-budget-gate, return-path-gate) — gap skills from PR #24–#28
- Added: Section 4.6 — Extended convergence table from 3 → 6 independent implementations (SwarmHarness arXiv:2605.28764, HarnessAPI arXiv:2605.22733, Harness-Bench arXiv:2605.27922)
- Added: References — SwarmHarness, HarnessAPI, Harness-Bench
- Updated: Section 3.1 Wave 5 — 7th regression check (merge-base auto-calculation in --pr mode, PR #30)
- Updated: Section 4 header — skill count 23 → 28 (24 fh-meta + 4 fh-commons)
- Updated: harvest-loop pipeline — Step 10 now lists 7-axis check
- Updated: Section 5.2 — added three new method rows (prompt-regression, mcp-circuit-breaker, token-budget-gate)
- Updated: Section 6 Conclusion — convergence evidence updated to six implementations
- Removed: Section 3.5 "Future Work: Harness Federation Sync" — superseded by agent-composer + context-bridge-dispatch; personal path mapping kept local-only

v0.6 (2026-05-28):

- Added: Section 2.5 — Empirical Validation: Multi-Model Review Convergence
- Added: Section 3.1 Wave 5 — Post-Fix Verification (Regression Guard)
- Added: harvest-loop Steps 0-a/0-b — Compression Loss Defense and Cross-Check Gate
- Added: harvest-loop Step 10 — Post-Fix Regression Guard
- Added: Section 3.5 — Future Work: Harness Federation Sync (preview, subsequently superseded)
- Improved: Section 2.1 — Added four new arXiv references (2605.25665, 2605.26302, 2605.27575, 2605.28065)
- Improved: Section 2.2 — Integrated Ning et al. two-pass meta-detector pattern
- Improved: Section 2.3 — Integrated Liu et al. compression aging mechanism
- Improved: Section 6 — Updated conclusion with multi-model convergence validation

Draft v1.0 — 2026-05-30.